IP 摄像头模块 —— 我们是怎样一步步做出来的

演讲稿 · 期末展示第一部分 · 第 05–15 页

ZHU Kecheng · SD5531 第 02 组 · 2026 年 11 月 6 日

> 约 6½ 分钟，内容与英文稿一一对应。按我们实际做的顺序来讲：几乎每个设计决定都来自一次失败的测试。
> 方括号 [ ] 里是提示（翻页或动作），不用念出来。
> 顺序上有一处不同：跟踪程序（第 13–14 页）其实是在“守宫移动才录”（第 11–12 页）之前做的。讲稿保持幻灯片顺序，在第 11 页用一句话说明。
> 所有测试用的都是绿色玩具蜥蜴，不是真守宫。

时间线

时间	这一步做了什么	学到了什么
10 月 30 日	接线；在家和学校各录第一段 60 秒	摄像头、PoE 和 ROCK 4C+ 能一起工作；1,200 帧全部解码，拷贝用 SHA-256 核对
11 月 5 日 02:51	对着 Mac 屏幕上的时钟测延迟	拍摄到 ROCK 0.23 秒；触发后才开录会丢 ⩾ 0.4 秒
03:07	内存环形缓冲 + 手动触发	预录有效；发现并修复了一个计时错误
03:24	帧差运动检测，在家用手测试	4 个事件，手出现前都有 3.7–4.65 秒
12:23–14:56	在学校第一次长时间运行	2.5 小时 86 段录像，没有一段是守宫
15:44	检测区域 + 整体变化规则 + 持续 0.5 秒	区域外 0 次误触发；区域内的手仍会触发
16:10	阈值 25 → 15	移动的玩具蜥蜴能录进去了
16:22 / 16:47	跟踪：先只用亮度，再加颜色	只框住尾巴 → 框住整只蜥蜴
17:47 / 18:04 / 18:45	守宫移动才录，三轮测试	只有手的片段 3 → 2 → 0

讲稿

第 05 页 IP 摄像头模块

约 20 秒

我是 Kecheng，负责视频部分。我不只是列出系统能做什么，而是按我们实际做的顺序，讲讲我们是怎么一步步做到的——因为几乎每个设计决定，都来自一次出了问题的测试。

第 06 页 第一步 · 先拿到画面

约 35 秒

第一步是 10 月 30 日，目标很简单：先拿到画面。摄像头接到 PoE 供电器上，一根网线同时送 48 伏电和视频；供电器、ROCK 4C+ 和 MacBook 都接在一台千兆交换机上。ROCK 从摄像头录了 60 秒，2560×1440，共 1,200 帧。我们把每一帧都完整解码，传到 Mac 后再用 SHA-256 核对。

这页有一个细节后来又碰到了。[指向右侧 USB-C] ROCK 是靠 Mac 的 USB-C 口供电的。它在学校两次突然停机，这条供电线路是我们的第一怀疑对象，但还没有证实。

第 07 页 第二步 · 先测量，再设计

约 40 秒

第二步是 11 月 5 日：先测量，再设计。摄像头同时发两路码流：一路全分辨率的主码流用来录像，一路小的子码流，分析起来很省资源。

测延迟时，我们让摄像头对着 Mac 屏幕上的毫秒时钟。[指向时钟照片] 画面被拍到之后 0.23 秒到达 ROCK。然后我们试了最直接的设计：有动静时才启动 FFmpeg。冷启动六次的结果是：第一帧总是晚约 0.4 秒，而且 FFmpeg 要大约 2 秒才开始写文件。所以“有动作才开录”一定会丢掉开头——而开头恰恰是学生最想看的那一刻。

第 08 页 第三步 · 先录下来，再决定留哪段

约 40 秒

于是有了第三步：先录下来，再决定留哪段。FFmpeg 一直不停，把主码流原样复制、不重新编码，切成 2 秒一段，放进内存里 60 秒的环形缓冲。事件结束时，把“第一次动作前 4 秒”到“最后一次动作后 5 秒”的分段拼起来。

我们对着时钟用手动触发来测，第一次就发现了一个错误：分段的时间是按“程序轮询看到它的时刻”算的，时间越来越偏，片段结尾被截短了。改成用文件的写入时间，加了一个测试，结尾就对了。[指向片段] 这段实拍里，蜥蜴在第 5.1 秒被拉动，片段里还能看到它被拉动之前的样子。目前已经有 97 个片段核对后转存到 Mac。

第 09 页 第四步 · 第一个检测器，第一个意外

约 35 秒

第四步是第一个检测器：第二个 FFmpeg 进程把子码流缩小到 160×90，每一帧和上一帧比较。在家用手测试是成功的：4 个事件，每个在手出现之前都留了 3.7 到 4.65 秒。

然后我们在学校连续跑了两个半小时，存下了 86 段录像——没有一段里有守宫。有人走动、手、灯光、摄像头自动曝光，全都被当成了运动。

第 10 页 第五步 · 告诉它该看哪里

约 35 秒

第五步是告诉它该看哪里。我们在网页上画出饲养箱的范围，范围外的变化一律不算。如果整幅画面有 30% 以上同时变化，就当作环境变化，不算运动。另外，运动要持续半秒才算。

测试结果：大约两分钟里，背景一直有人走动，一段都没录；一次 31.4% 的整幅画面变化也被忽略了。[指向第三张图] 但是手伸进盒子里，还是会录。我们还发现了相反的问题——玩具蜥蜴动了经常录不到——所以把阈值从 25 降到了 15。

第 11 页 第六步 · 只在守宫本身移动时才录

约 35 秒

这引出了最关键的第六步：不再是“有变化就录”，而是“守宫本身动了才录”。这需要一个跟踪程序——它是在这一步之前做好的，两页之后我再讲它怎么工作。

现在区域规则只写进日志。只有被锁定的守宫真的移动，才开始录：整个框在一秒内至少平移 2 个像素，而且两侧边要朝同一个方向移动。影子只会让一侧边变长，所以不算。

第 12 页 第六步 · 实验室三轮测试

约 40 秒

我们分三轮测试，每一次失败都变成了一条新规则。第一轮，17:47：录到 3 段只有手的片段。手让蜥蜴身上的像素闪动，所以我们要求中心真正移动，并且持续半秒。

第二轮，18:04：还有 2 段只有手的片段。手的影子和蜥蜴连成一块，把框拉走了，所以我们加了“整个框平移”、重叠检查和大小限制。第三轮，18:45：手在区域里停了 8 秒，旧的区域规则触发了 87 帧，但一段都没有录——而三段真正的蜥蜴片段全部保留了。

第 13 页 跟踪程序怎样找到守宫

约 30 秒

这是我们依赖的跟踪程序。它先用静止的画面学出背景，标出和背景不同的像素，把它们连成块，然后锁定那个在动的块。

第一次实机运行时，框只框住了尾巴。原因在亮度：绿色蜥蜴和桌面几乎是同一个灰度——157 对 156 到 186。但在颜色通道上两者差得很远，所以我们加入了颜色。整个程序是纯 Python，每帧约 8 毫秒，优化前是 63 毫秒。

第 14 页 跟踪结果

约 25 秒

加入颜色后，抽查 6 帧里有 5 帧框住了整只蜥蜴；蜥蜴静止 55 秒，没有被误判为“丢失”——第一版在静止约 100 秒后就判丢失了。一个局限：蜥蜴被快速拿走时，框可能在原地停留大约 2 秒。

第 15 页 现在做到哪一步

约 35 秒

所以一步一步下来，我们从“86 段录像里没有一段是守宫”，做到了“0 段只有手的片段”，而且每段录像都保留了第一次动作前 4 到 6 秒。

还没做的：真守宫——所有测试都用的是玩具蜥蜴；用摄像头红外模式的夜间录像；以及超过几个小时的长时间运行。下一步：在真实饲养箱里测试，做夜间测试，给 ROCK 配独立电源后跑 24 小时，并把视频片段和热成像事件关联起来——这就引出下一部分：热成像传感器。**[交给下一位同学]**

可能被问到的问题

为什么视频不用 ESP32-C3?	它没有以太网，内存放不下一帧 2560×1440 的画面，也不能解码 H.264，所以录像放在 ROCK 4C+ 上。
真守宫能用吗?	还没测。所有测试都用玩具蜥蜴，进真实饲养箱测试是下一步的第一项。
第 08、10、12、14 页的画面是实时的吗?	不是。是回放：把录下的片段重新送进同一套检测和跟踪程序，逐帧画框。

事实依据：project-camera/experiments（2026 年 10 月 30 日与 11 月 5 日）。数字与 Figma “Final PPT” 第 05–15 页一致。